

Sales Pipeline: write-up

What I built and why

One screen, two ways to look at the same deals:

- **Table (the default).** A virtualized, keyboard-driven list under a **stage strip** showing every stage's live count and ₹ value. It's built for triage, sorting, filtering and moving thousands of deals at once.
- **Board.** Five columns of cards (New Lead → Negotiation), with Won and Lost as drop zones. It's built for seeing where deals sit and moving them with the mouse. Press V to switch; the app remembers your choice.

Both views show the same filtered, sorted, stable list, with the same selection, the same keys and the same save engine. The board is just that list grouped by stage, so nothing needs to be kept in sync between the two.

Why both, with Table as the default: a board is what salespeople expect, and it's the best overview. But a 10,000-card column can't be scanned, you can't pick 3,000 deals across columns, and dragging is the opposite of "I can't use a mouse for long". So the table does the heavy work, and the board shines on smaller scopes, like **My focus** (~380 deals), and for quick drags. Every column has **"Open in table ↗"** for the moment a job gets big.

What the team said	What the screen does
"I don't know which deals to work on today"	My focus (in both views): my deals that are overdue or closing within 7 days, plus late-stage or high-value deals that have gone quiet. Ranked, and each one says <i>why</i> ("Close date passed 12d ago").
"It didn't save and nobody told me"	Every change stays visibly unconfirmed until the server confirms it. Failures stay on screen until I act.
"Deals keep jumping around"	Teammates' edits update rows and cards in place. Nothing reorders, and no card changes column because of someone else.
"Move thousands of old deals to Lost"	Filter (e.g. <i>No activity 90+ days</i>), then ⌘A selects all matching deals, not just the visible ones, then 7 and a reason key. One dialog, then a background job with progress, Stop and Undo.
"I can't use a mouse for long"	Every action has a key (J/K, X, 1–7, [], M, Z). On the board, arrows move between columns and ⬅️/➡️ carries a card to the next stage. Dragging is an extra, never the only way.

Key UX decisions and what I rejected

- **Optimistic updates that show their state.** Blocking until a save finishes (0.3–1.5 s each) makes triage painful. Silent optimism is the bug the team reported. Rows and cards show *Saving...* → *Didn't save, retrying* → *Not saved*, and a header pill always answers "is everything saved?"
- **Undo instead of confirming every move.** Confirmation only appears for more than 20 deals, and it shows the count, the ₹ value and the stages they come from.
- **Drops change the stage, not the position.** Cards are always sorted. I rejected Trello-style

manual ordering because a shared pipeline has no single "right" order. That let me build drag-and-drop in about 150 lines without a library: drop targets are whole stages, so it works with virtualized lists, and every drop takes the same path as the keyboard.

- **Won/Lost as drop zones, not columns.** Closed deals are an outcome, not a list to browse. That lets 5 columns fit on a laptop without sideways scrolling.
- **A stable list instead of live sorting or locking.** Live sorting is what makes deals jump. Locking blocks teammates.

Failed saves

- **One request per deal, latest intent wins.** If I change my mind mid-save, the new choice is sent as a follow-up rather than racing the first request.
- **Automatic retries, then an honest rollback.** Transient failures retry twice with backoff, and the row or card says so. After that, the deal returns to the **latest server state**, not a stale snapshot, in red with *Retry / Discard*. A sticky toast, the header pill and a drawer of every unsaved change (A) stay until I act. Closing the tab with unsaved work shows a warning.
- **Idempotency keys.** A retry reuses the same mutation id, and the server deduplicates it, so a save whose reply was lost is never applied twice.
- **Offline.** Saves pause and changes show as *waiting*. They're sent when the connection comes back.
- **Bulk moves** go in chunks of 500, three at a time. One lane always stays free, so a single move never waits behind a 4,000-deal job. Partial failures are reported per deal.

Teammates' changes

- **Pending changes stay on top.** The client keeps the server's version of each deal plus an overlay of my pending changes. Pushed events update the server copy underneath. Version numbers drop stale and echoed events. Events are applied in 200 ms batches, one render per batch.
- **Nothing moves because of someone else.** Edited rows and cards flash briefly ("Rahul updated the value"). If a teammate moves a deal, it **stays where I saw it**, greyed out ("Moved to Won by Rahul"). On the board, the destination column shows "+3 from teammates · Refresh (U)". Focus and selection follow the deal, not its position.
- **Conflicts are explicit.** Each move carries the stage I saw. If a teammate got there first, the server refuses rather than overwriting them, and I get "*Divya moved Acme to Won first. Keep Won / Move to Lost anyway.*"
- **Stale picks are skipped.** If a teammate changes a deal after I selected it, the move skips it and tells me why.

Closing deals and bulk edits

- **Lost needs a reason; Won offers a note.** Every way into Won/Lost (key, menu,], drag) opens one small dialog: seven fixed reasons on keys 1-7, and "Other" needs a note. A fixed list, not free text, because the point is to count reasons later. A bulk close uses the same dialog as the bulk confirmation, so 3,941 stale deals are still one step: 7, 4, Enter.
- **One rule, checked twice.** The dialog and the server share `closeProblem()`. The server refuses a bad request as `invalid`. That isn't retried, since resending can't fix it, and the row says "*Refused: Pick a reason*".
- **Details travel with the move** in the same intent and request, so a retry or a lost reply can never save the stage without its reason. Reopening a deal clears them, and undo puts the old

reason back.

- **Bulk edit (E)** covers owner and close date (set it, move each by N days, or end of next quarter): the fields reps change in bulk. Value stays per deal.
- **Each request checks only what it changes.** Each field carries the value I saw, and the server refuses only if *that field* changed, so reassigning never clashes with a teammate moving the deal. A request is all-or-nothing, because half an intent is a state nobody asked for. A move and an edit made before the first save merge into one request.

Performance

Measured on a production build in a sandboxed Chromium (slower than a typical laptop): moving focus ~1.5 ms; switching to All open (37k deals) 60-90 ms; search 45-70 ms; Table ⇌ Board ~35 ms; an optimistic move or bulk edit of 37k deals 160-210 ms; 30 teammate edits/s on screen at 56-58 fps. The heap is about 18 MB, and the DOM holds about 30 rows or 55 cards at any size. That comes from virtualization, per-item selectors on immutable records, batched events with a list snapshot that isn't re-sorted per event, and cached sort keys. **Tests:** 28 unit tests (sync engine, close details, field edits), 6 Playwright smoke tests, and a test that keeps the code split honest: `src/` has `client/`, `server/` (the fake backend) and `shared/` (the API contract), and the client reaches the server only through `shared/api.ts`.

With more time

1. **Scale.** Filtering in a Web Worker; beyond about 500k deals, server-side pagination, search and counts.
2. **Survive a reload.** Unsaved changes in IndexedDB, coordinated between tabs. **Server-side bulk jobs** with pushed progress.
3. **Prevent conflicts.** Presence ("Rahul is viewing this"), and inline editing of more fields under the same per-field checks.
4. **Use the reasons.** A lost-reason breakdown by stage and owner. Snooze and configurable scoring for My focus.
5. **Accessibility pass** with VoiceOver and NVDA, especially for the board.

Known limits

- **All 50k deals are held in memory.** Fine at this size, wrong at 10×.
- **The board is best-effort for screen readers, and drag is mouse-only.** The table is the accessible primary view, and every drag has a key.
- **Undo skips deals a teammate has changed since. If a teammate closes a deal first, their reason stands.**
- **The fake push channel never drops messages**, so there's no resync for missed events. **Tested in Chromium only.**
- **Built with AI assistance (Claude).** The decisions and trade-offs above are mine to defend.